

CHAPTER 1 – COMPUTER ABSTRACTIONS AND TECHNOLOGY PATTERSSON/HENNESSY

Luiz Paulo de Assis Barbosa

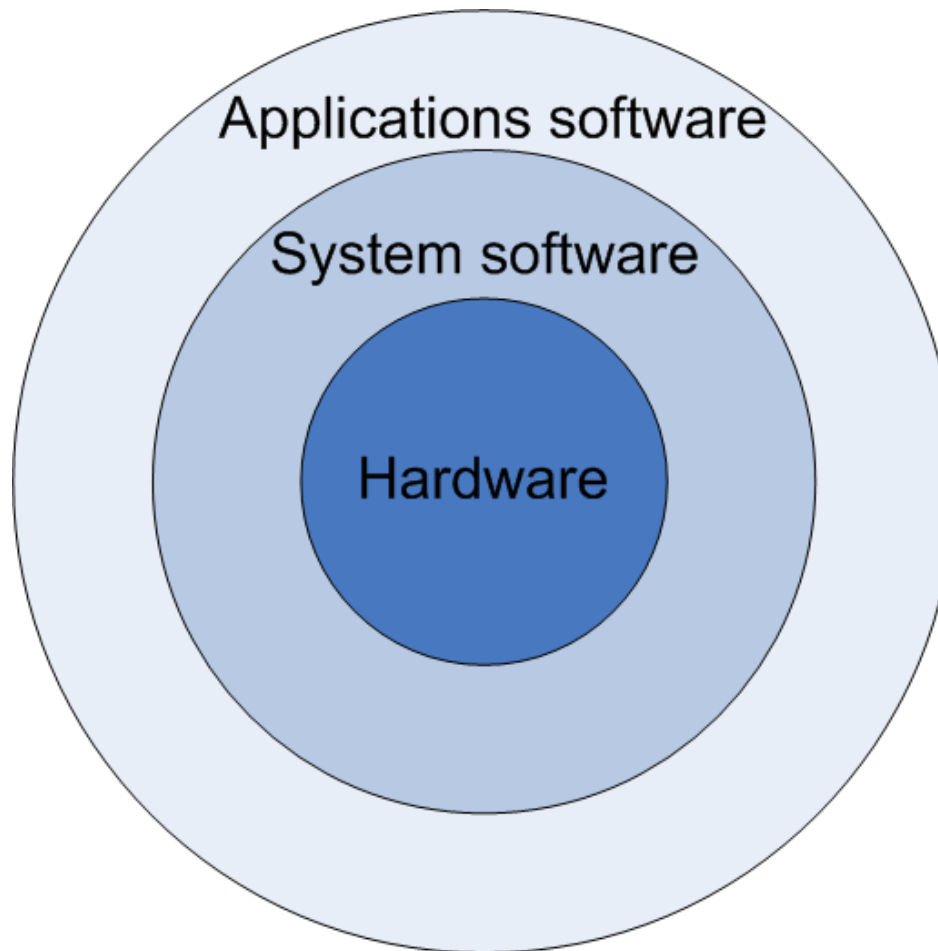
1.1 8 Grandes Ideias

2

- ❑ Projetar para a Lei de Moore
- ❑ Usar abstração para simplificar o projeto
- ❑ Tornar o caso mais comum o mais rápido
- ❑ Performance via paralelismo
- ❑ Performance via pipeline
- ❑ Performance via predição
- ❑ Hierarquia de memórias
- ❑ Confiabilidade via redundância

1.2 – Abaixo de seu programa

3



1.2 – Abaixo de seu programa

4

- Dois dos principais software de sistema são: sistema operacional (SO) e compilador.
- SO – promove a interface entre os programas de usuário e o hardware e provê uma variedade de serviços e funções de supervisão. Entre as funções mais importantes temos:
 - ▣ Tratar das operações de entrada e saída básicas;
 - ▣ Alocar armazenamento e memória;
 - ▣ Prover o compartilhamento da máquina entre múltiplas aplicações simultâneas de forma protegida, ou seja, sem que uma interfira com a outra de forma indesejada.

1.2 – Abaixo de seu programa

5

- Compiladores – realizam uma função vital, a tradução de um programa em linguagem de alto nível em instruções que o hardware pode executar.

1.2 – Abaixo de seu programa

6

- Para que haja comunicação entre componentes de hardware é necessário a troca de sinais elétricos entre eles.
- Os sinais mais fáceis de serem compreendidos pelos computadores são, **ligado** e **desligado**, desta forma o “alfabeto” da máquina possui apenas dois símbolos **0** e **1**.
- Os computadores são escravos que obedecem aos nossos comandos. Os comandos são denominados **instruções**.

1.2 – Abaixo de seu programa

7

- Instruções são apenas um conjunto de bits que o computador entende e obedece, podem ser visualizadas como números binários. Por exemplo:

```
1000110010100000
```

- Diz ao computador para adicionar dois números.

1.2 – Abaixo de seu programa

8

- Os primeiros programadores se comunicavam com os computadores por meio dos números, porém logo foi inventada uma notação mais próxima da forma humana de pensar.

```
add A, B
```

- No princípio a tradução dessa notação para binário era feita à mão, no entanto, pode-se usar o computador para auxiliar em sua própria programação e desta forma foram escritos programas para automatizar a tarefa de tradução.

1.2 – Abaixo de seu programa

9

- O primeiro desses programas foi denominado ***Assembler (montador)***.
- A linguagem simbólica utilizada foi batizada de linguagem ***Assembly***.

```
add A, B
```

- Enquanto que a linguagem binária é denominada ***linguagem de máquina***.

```
1000110010100000
```

1.2 – Abaixo de seu programa

10

- Programa em linguagem de alto nível (C) para Assembly (MIPS)

```
swap(int v[], int k)
{
    int temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```



```
swap:
    multi $2, $5, 4
    add    $2, $4, $2
    lw     $15, 0($2)
    lw     $16, 4($2)
    lw     $16, 0($2)
    lw     $15, 4($2)
    jr     $31
```

1.2 – Abaixo de seu programa

11

- Programa em Assembly (MIPS) para linguagem de máquina binária

swap:

```
multi $2, $5, 4
add   $2, $4, $2
lw    $15, 0($2)
lw    $16, 4($2)
lw    $16, 0($2)
lw    $15, 4($2)
jr    $31
```



```
00000000101000010000000000011000
00000000000110000001100000100001
10001100011000100000000000000000
100011001111001000000000000000100
10101100111100100000000000000000
101011001100010000000000000000100
0000001111100000000000000000001000
```

1.4 – Performance

12

- Quando dizemos que um computador tem melhor performance que outro, o que isso significa?
- Fazendo uma analogia com a aviação civil, se definirmos performance em termos de velocidade, distância, ou capacidade qual o melhor avião?

Avião	Capacidade (passageiros)	Distância de cruzeiro (milhas)	Velocidade de cruzeiro (m.p.h)	Vazão de passageiros (passageiros x m.p.h)
Boeing 777	375	4630	610	228.750
Boeing 747	470	4150	610	286.700
BAC/Sud Concorde	132	4000	1350	178.200
Douglas DC-8-50	146	8720	544	79.424

1.4 – Performance

13

- Se queremos levar um passageiro de um ponto a outro no menor tempo possível, o melhor é o que tem a maior velocidade de cruzeiro (em verde).
- Se o objetivo é levar 450 passageiros de um ponto a outro o outro, o que tem maior vazão é o melhor (em amarelo).

Avião	Capacidade (passageiros)	Distância de cruzeiro (milhas)	Velocidade de cruzeiro (m.p.h)	Vazão de passageiros (passageiros x m.p.h)
Boeing 777	375	4630	610	228.750
Boeing 747	470	4150	610	286.700
BAC/Sud Concorde	132	4000	1350	178.200
Douglas DC-8-50	146	8720	544	79.424

1.4 – Performance

14

- Um usuário de computador pessoal dirá que o melhor computador é aquele que termina uma tarefa primeiro, em outras palavras, é aquele que apresenta o **menor tempo de resposta** ou **tempo de execução**.
- Já um administrador de *datacenter* dirá que o melhor computador é aquele que completa mais tarefas durante um período de tempo, ou seja, está interessado em **aumentar a vazão (*throughput*)** ou **largura de banda (*bandwidth*)**.

1.4 – Performance

15

- ❑ **Vazão e tempo de resposta**
- ❑ As seguintes mudanças em um computador aumentam a vazão, reduzem o tempo de resposta, ou afeta ambos?
 - ▣ Trocar o processador por uma versão mais rápida;
 - ▣ Acrescentar processadores a um sistema que suporta múltiplos processadores para tarefas distintas.

1.4 – Performance

16

- Para o primeiro caso, quase sempre *diminuir* o **tempo de resposta** melhora a vazão.
- Para o segundo caso, nenhuma tarefa individual será realizada em menor tempo, portanto apenas a vazão aumenta.
- No entanto, se a demanda por processamento no segundo caso era quase igual a vazão do sistema original, poderiam existir filas de tarefas. Neste caso, o tempo de resposta também poderia melhorar devido a redução do tempo de espera na fila.

1.4 – Performance

17

- Focando no parâmetro ***tempo de resposta***, temos:
 - ▣ Para ***maximizar*** a performance temos que ***reduzir*** o ***tempo de resposta***.
 - ▣ Logo, podemos relacionar performance e tempo de execução em um computador X:

$$Performance_X = \frac{1}{tempo\ de\ execução_X}$$

1.4 – Performance

18

- O que significa que para dois computadores X e Y, se a performance de X é maior que a de Y, temos:

$$Performance_X > Performance_Y$$

$$\frac{1}{tempo\ de\ execução_X} > \frac{1}{tempo\ de\ execução_Y}$$

$$tempo\ de\ execução_Y > tempo\ de\ execução_X$$

1.4 – Performance

19

- Para relacionar a performance de dois computadores quantitativamente, normalmente usamos a expressão X é n vezes mais rápido que Y , onde n é dado por:

$$\frac{Performance_X}{Performance_Y} = \frac{tempo\ de\ execução_Y}{tempo\ de\ execução_X} = n$$

1.4 – Performance

20

□ Performance relativa

- *Ex. Se o computador A executa um programa em 10s e o computador B executa o mesmo programa e 15s, A é quantas vezes mais rápido que B?*

$$\frac{Performance_A}{Performance_B} = \frac{tempo\ de\ execução_B}{tempo\ de\ execução_A} = \frac{15}{10} = 1,5 = n$$

- *Podemos dizer também que B é 1,5 mais lento que A, ou seja:*

$$Performance_B = \frac{Performance_A}{1,5}$$

1.4 – Performance

21

- Um computador C é 4 vezes mais rápido que um computador B, o qual executa uma dada aplicação em 28s. Quanto tempo o computador C leva para executar a mesma aplicação?

$$\frac{Performance_X}{Performance_Y} = \frac{tempo\ de\ execução_Y}{tempo\ de\ execução_X} = n$$

$$\frac{tempo\ de\ execução_B}{tempo\ de\ execução_C} = \frac{28}{tempo\ de\ execução_C} = 4$$

$$tempo\ de\ execução_C = \frac{28}{4} = 7s$$

1.4 – Performance

22

□ Medindo Performance

- ▣ Tempo de resposta ou tempo transcorrido (*elapsed time*)
 - Refere-se ao tempo total para executar uma tarefa, incluindo acesso a disco, acesso à memória, operações de entrada e saída, *overhead* do SO, enfim, tudo.
- ▣ Tempo de execução de CPU ou tempo de CPU
 - É o tempo que a CPU gasta computando uma tarefa específica, e não inclui os tempos de espera por entrada e saída ou gastos executando outros programas.

1.4 – Performance

23

- O tempo de CPU pode ser dividido em:
 - ▣ Tempo de de CPU de usuário
 - Tempo gasto pela CPU no programa do usuário.
 - ▣ Tempo de CPU de sistema
 - Tempo gasto pela CPU realizando serviços do SO em favor do programa do usuário.
 - Difícil de distinguir de maneira precisa entre os dois.
 - ▣ Será usado o termo **performance de sistema** para designar a perfromance baseada no **tempo trasecorrido** e o termo **performance de CPU** para o caso baseado no **tempo de CPU**.

1.4 – Performance

24

- ❑ **Entendendo a performance de programa**
- ❑ Quase todos os computadores são construídos usando um **relógio (clock)** que determina quando ocorrem os eventos no *hardware*.
- ❑ Estes intervalos de tempo discretos são chamados **ciclos de clock**. O **período** é a quantidade de tempo gasta para realizar um ciclo de clock completo (ex. 250ps). A **frequência ou taxa de relógio (clock rate)** (ex. 4GHz) corresponde ao inverso do período.

1.4 – Performance

25

□ Performance de CPU e seus fatores

$$\text{tempo de CPU de um programa} = \frac{\text{Ciclos de CPU do programa}}{\text{período}}$$

- Ou alternativamente, devido à relação entre período e frequência:

$$\text{tempo de CPU para um programa} = \frac{\text{Ciclos de CPU para o programa}}{\text{frequência}}$$

1.4 – Performance

26

- Exemplo (melhorando performance)
 - ▣ Um computador A possui um clock de 2GHz e executa um programa P em 10s.
 - ▣ O objetivo é construir um computador B que execute P em 6s.
 - ▣ Foi determinado que é possível aumentar substancialmente a frequência f , porém ao custo de que o computador B necessitará de 1,2 vezes mais ciclos de clock que o computador A para executar P.

1.4 – Performance

27

□ Solução:

- ▣ Achar o número de ciclos que A usa para executar P:

$$\text{tempo de CPU}_A = \frac{\text{Ciclos de CPU}_A}{\text{frequência}_A}$$

$$\text{Ciclos de CPU}_A = \text{tempo de CPU}_A \times \text{frequência}_A$$

$$\text{Ciclos de CPU}_A = 10\text{s} \cdot 2 \times 10^9 \frac{\text{ciclos}}{\text{s}} = 20 \times 10^9 \text{ ciclos}$$

1.4 – Performance

28

- Continuação da solução:
 - ▣ Determinar a frequência do clock que B precisa para executar P em 6s, usando:

$$frequência_B = \frac{Ciclos\ de\ CPU_B}{tempo\ de\ CPU_B}$$

$$frequência_B = \frac{Ciclos\ de\ CPU_B}{tempo\ de\ CPU_B} = \frac{1,2 \cdot Ciclos\ de\ CPU_A}{tempo\ de\ CPU_B} = \frac{1,2 \cdot 20 \times 10^9\ ciclos}{6\ s} = 4 \times 10^9 \frac{ciclos}{s} = 4GHz$$

1.4 – Performance

29

□ Performance de instrução

- ▣ O tempo de execução de um programa depende do número de instruções que compõem o programa.
- ▣ Desta forma o número de ciclos de CPU para um dado programa pode ser calculado por:

$$\text{Ciclos de CPU} = \frac{\text{Número de instruções do programa}}{\text{Número médio de ciclos de clock por instrução}}$$

1.4 – Performance

30

- **Performance de instrução (Continuação)**
 - ▣ O número **médio** de ciclos de clock por instrução pode ser abreviado por **CPI** (*Clock cycles Per Instruction*)
 - ▣ A métrica CPI provê uma maneira de comparar diferentes implementações de um mesmo conjunto de instruções, **já que o número de instruções executadas por um programa será o mesmo.**

1.4 – Performance

31

- Exemplo (usando a equação de performance)
 - ▣ Suponha que temos 2 implementações do mesmo ISA (Instruction Set Architecture);
 - ▣ Um computador A possui um período de ciclo de clock de 250ps e uma CPI de 2,0 para um programa P;
 - ▣ E um outro computador B possui um período de ciclo de clock de 500ps e uma CPI de 1,2 para P;
 - ▣ Qual computador, A ou B, é o mais rápido para executar P?
 - ▣ E quanto mais rápido ele é em relação ao outro?

1.4 – Performance

32

- Solução:
- Façamos I corresponder ao número de instruções de P. Agora podemos calcular os ciclos de CPU para A e B em função de I , usando:

$$\text{Ciclos de CPU}_A = I \times \text{CPI}_A = 2 \cdot I$$

$$\text{Ciclos de CPU}_B = I \times \text{CPI}_B = 1,2 \cdot I$$

1.4 – Performance

33

- Os tempos de CPU para A e B podem ser determinados usando:

$$\text{tempo de CPU}_A = \frac{\text{Ciclos de CPU}_A}{\text{frequência}_A} = \text{Ciclos de CPU}_A \times \text{período}_A$$

$$\text{tempo de CPU}_A = 2 \cdot I \times 250 \text{ ps} = 500 \cdot I \text{ ps}$$

- ▣ De forma análoga:

$$\text{tempo de CPU}_B = 1,2 \cdot I \times 500 \text{ ps} = 600 \cdot I \text{ ps}$$

- ▣ Ou seja, A é mais rápido que B.

1.4 – Performance

34

- Utilizando a relação para performance relativa:

$$\frac{\text{Performance de CPU}_A}{\text{Performance de CPU}_B} = \frac{\text{tempo de execução}_B}{\text{tempo de execução}_A} = n$$

$$\frac{\text{Performance de CPU}_A}{\text{Performance de CPU}_B} = \frac{600 \cdot I \text{ ps}}{500 \cdot I \text{ ps}} = 1.2$$

$$\text{Performance de CPU}_A = 1,2 \cdot \text{Performance de CPU}_B$$

- Logo, A é 1,2 vezes mais rápido que B.

1.4 – Performance

35

□ A equação clássica da performance de CPU

$$\textit{tempo de CPU} = I \times CPI \times \textit{período} = \frac{I \times CPI}{\textit{frequência}}$$

- Ela é particularmente útil, pois separa os 3 fatores chaves que afetam a performance.

1.4 – Performance

36

- Exemplo (comparando segmentos de código)
 - ▣ Um projetista de compiladores está tentando decidir entre 2 sequências de código para um determinado computador.
 - ▣ Ele sabe que:

CPI para cada classe de instrução			
Classe	A	B	C
CPI	1	2	3

1.4 – Performance

37

- Para uma linguagem de alto nível L as sequências de instruções em consideração levam a duas contagens de instruções distintas, a saber:

Número de instruções para cada classe de instrução			
Classe	A	B	C
Seq. 1	2	1	2
Seq. 2	4	1	1

- Qual sequência de código executa mais instruções? Qual a mais rápida? Qual é a CPI para cada sequência?

1.4 – Performance

38

□ Solução:

▣ Qual sequência de código executa mais instruções?

- A sequência 1 executa $2+1+2 = 5$ instruções,
- enquanto a sequência 2 executa $4+1+1 = 6$ instruções

▣ Qual a mais rápida?

- Para determinar isso precisamos descobrir os ciclos de CPU para cada sequência:

$$\text{Ciclos de CPU} = \sum_{i=1}^n (CPI_i \times I_i)$$

1.4 – Performance

39

□ Logo,

$$\text{Ciclos de CPU}_1 = (2 \times 1) + (1 \times 2) + (2 \times 3) = 2 + 2 + 6 = 10 \text{ ciclos}$$

$$\text{Ciclos de CPU}_2 = (4 \times 1) + (1 \times 2) + (1 \times 3) = 4 + 2 + 3 = 9 \text{ ciclos}$$

□ Portanto, a sequência 2 é a mais rápida, mesmo tendo um numero de instruções maior, o que nos diz que a $CPI_2 < CPI_1$.

1.4 – Performance

40

□ Qual é a CPI para cada sequência?

■ Sabemos que:

$$\text{Ciclos de CPU} = I \times \text{CPI}$$

$$\text{CPI} = \frac{\text{Ciclos de CPU}}{I}$$

■ Assim,

$$\text{CPI}_1 = \frac{\text{Ciclos de CPU}_1}{I_1} = \frac{10}{5} = 2$$

■ Analogamente,

$$\text{CPI}_2 = \frac{\text{Ciclos de CPU}_2}{I_2} = \frac{9}{6} = 1,5$$

1.4 – Performance

41

- Uma dada aplicação (A) em Java roda em 15s num processador de um desktop. Um novo compilador Java é lançado e requer apenas 60% do número de instruções do compilador antigo para A.
- Infelizmente, ele aumenta a CPI por um fator de 10%.
- Quanto rápido podemos esperar que a aplicação rode usando o novo compilador?

1.4 – Performance

42

□ Alternativas:

a) $\frac{15 \times 0,6}{1,1} = 8,2 \text{ s}$

b) $15 \times 0,6 \times 1,1 = 9,9 \text{ s}$

c) $\frac{15 \times 1,1}{0,6} = 27,5 \text{ s}$

□ Demonstre a partir da relação das variáveis do problema qual das alternativas é a correta.

1.4 – Performance

43

□ Dados:

$$t_{CPU1} = 15 \text{ s}$$

$$I_2 = 0,6 \cdot I_1$$

$$CPI_2 = 1,1 \cdot CPI_1$$

□ Sabemos:

$$t_{CPU} = \frac{I \times CPI}{f} = I \times CPI \times T$$

□ Compilador antigo (1)

$$15 = \frac{I_1 \times CPI_1}{f}$$

□ Compilador Novo (2)

$$t_{CPU2} = \frac{I_2 \times CPI_2}{f}$$

$$t_{CPU2} = \frac{0,6 \cdot I_1 \times 1,1 \cdot CPI_1}{f}$$

$$t_{CPU2} = \frac{0,6 \times 1,1 \cdot (I_1 \times CPI_1)}{f}$$

1.4 – Performance

44

- Isolando f e igualando as equações:

$$f = \frac{I_1 \times CPI_1}{15}$$

$$f = \frac{0,6 \times 1,1 \cdot (I_1 \times CPI_1)}{t_{CPU2}}$$

$$\frac{I_1 \times CPI_1}{15} = \frac{0,6 \times 1,1 \cdot (I_1 \times CPI_1)}{t_{CPU2}}$$

- Resolvendo para t_{CPU2}

$$t_{CPU2} = \frac{15 \times 0,6 \times 1,1 \cdot (I_1 \times CPI_1)}{I_1 \times CPI_1}$$

$$t_{CPU2} = 15 \times 0,6 \times 1,1 = 9,9 \text{ s}$$

1.6 – Uniprocessadores para Multiprocessadores

45

- A relação entre frequência de operação e potência dissipada nos processadores forçou uma mudança dramática no projeto de microprocessadores.
- A mudança de uniprocessadores (*single core*) para multiprocessadore (*multicore*).
- Em vez de diminuir o tempo de resposta de um único programa rodando em um único processador os microprocessadores passaram a oferecer mais de um processador (*core*) por pastilha.
- Deslocando os benefícios de um novo processador do tempo de resposta para a vazão.

1.6 – Uniprocessadores para Multiprocessadores

46

- Antes os programadores podiam confiar nas inovações de *hardware*, arquitetura e compiladores que faziam dobrar a performance dos seus programas a cada 18 meses, sem a necessidade de reescrever seus programas.
- Atualmente, para se obter uma melhora significativa no tempo de resposta é necessário reescrever os programas para tirar vantagens dos múltiplos processadores disponíveis.

1.6 – Uniprocessadores para Multiprocessadores

47

- Escrever programas que exploram paralelismo explicitamente é programação voltada a performance o que dificulta a atividade de programação, pois:
 - ▣ O programa tem que resolver o problema corretamente e tem que ser rápido.
 - ▣ A aplicação tem que ser dividida para que cada processador receba aproximadamente a mesma quantidade de trabalho ao mesmo tempo, de forma que o *overhead* de agendamento e coordenação não anule os benefícios do paralelismo.

1.6 – Uniprocessadores para Multiprocessadores

48

- Fazendo uma analogia com a escrita de um artigo de jornal, no qual 8 repórteres estão trabalhando na mesma estória.
- Para termos ganho em velocidade:
 - ▣ Todos têm que ter trabalho ao mesmo tempo;
 - ▣ Se um atrasar degrada a performance;
 - ▣ Não pode haver muita comunicação entre eles para que o artigo seja escrito, nem uma parte que dependa das outras para ser escrita, por exemplo, a conclusão;

1.6 – Uniprocessadores para Multiprocessadores

49

- Desafios da programação paralela:
 - ▣ Agendamento de tarefas;
 - ▣ Balanceamento de carga;
 - ▣ Redução da comunicação;
 - ▣ Redução do *overhead* de sincronização.

1.8 – FALÁCIAS E ARMADILHAS

50

- Armadilha:
 - ▣ Esperar que a melhoria de um aspecto do computador aumente a performance geral por uma quantidade proporcional ao tamanho da melhoria.
- Suponha que um programa roda em 100s num computador, com as operações de multiplicação sendo responsáveis por 80s desse tempo.
- Quanto eu tenho que melhorar a velocidade da operação de multiplicação para que o programa rode 5 vezes mais rápido?

1.8 – FALÁCIAS E ARMADILHAS

51

- Lei de Amdahl – calcula o tempo de execução de um programa após uma melhoria.

$$t_{\text{exec. depois da melhoria}} = \frac{t_{\text{exec. afetado pela melhoria}}}{\text{Quantidade de melhoria}} + t_{\text{exec. não afetado}}$$

- Para o problema em particular:

$$\frac{100 \text{ segundos}}{5} = 20 \text{ segundos} = \frac{80 \text{ segundos}}{n} + 20 \text{ segundos}$$

$$0 = \frac{80 \text{ segundos}}{n}$$

1.8 – FALÁCIAS E ARMADILHAS

52

□ Conclusão:

- Não existe como atingir uma performance geral 5 vezes melhor apenas melhorando a performance das operações de multiplicação, se elas representarem apenas 80% da carga de trabalho.

□ Falácia – Computadores em baixa utilização gastam menos potência.

- Estudos mostram que mesmo operando em 10% da carga máxima, ainda são gastos 66,6% da potência consumida em carga máxima.

1.8 – FALÁCIAS E ARMADILHAS

53

- Armadilha – Usar um subconjunto da equação de performance como métrica de performance.
- Uma métrica alternativa ao tempo é o MIPS.
- MIPS (Million Instructions Per Second), não confundir com o MIPS processador.

$$MIPS = \frac{I}{t_{exec.} \times 10^6}$$

1.8 – FALÁCIAS E ARMADILHAS

- Problemas ao usar MIPS para comparar computadores:
 - ▣ Especifica a taxa de instruções, mas não leva em conta as capacidades de cada instrução;
 - ▣ Computadores com ISAs diferentes não podem ser comparados usando MIPS, a contagem de instruções seria diferente;
 - ▣ A métrica MIPS é dependente do programa e um computador pode apresentar vários MIPS para programas diferentes;
 - ▣ Se um novo programa executa mais instruções, mas cada instrução é mais rápida, o MIPS pode variar de forma independente da performance.

1.8 – FALÁCIAS E ARMADILHAS

55

- Relacionamento entre MIPS, frequência (*clock rate*) e CPI:

$$MIPS = \frac{I}{t_{exec.} \times 10^6} = \frac{I}{\frac{I \times CPI}{f} \times 10^6} = \frac{f}{CPI \times 10^6}$$

- Exemplo: Qual computador possui o maior MIPS? E qual é o mais rápido?

Métrica	Computador A	Computador B
I	10 bilhões	8 bilhões
f	4GHz	4GHz
CPI	1,0	1,1

1.8 – FALÁCIAS E ARMADILHAS

56

□ Calculando MIPS:

$$MIPS_A = \frac{4 \times 10^9}{1 \times 10^6} = 4 \times 10^3 = 4.000 \text{ MIPS}$$

$$MIPS_B = \frac{4 \times 10^9}{1,1 \times 10^6} = 3,64 \times 10^3 = 3.640 \text{ MIPS}$$

□ Calculando $t_{exec.}$

$$t_{exec. A} = \frac{10 \times 10^9 \times 1,0}{4 \times 10^9} = 2,5 \text{ s}$$

$$t_{exec. B} = \frac{8 \times 10^9 \times 1,1}{4 \times 10^9} = 2,2 \text{ s}$$